Taller de Segmentación — Guía de estudio y sustentación

Oct 12, 2026 · @Manuel Alejandro Gruezo

Cómo usar esta guía

El taller compara cuatro formas de segmentar imágenes (U-Net++, Mask R-CNN, YOLO-seg y SAM) y termina con un reto: un pipeline YOLO → SAM que superó a YOLO solo con IoU 0.9523 frente a 0.9419. Esta guía explica cada parte del notebook `taller_segmentacion.ipynb` desde cero, con su código, sus fórmulas y lo que significan sus resultados.

Léela en orden la primera vez. Cada sección sigue el mismo patrón: qué hace la celda, el código clave, la fórmula si la hay y cómo interpretar lo que imprimió o dibujó. Al final están las preguntas probables del profesor y el guion de sustentación.

Integrantes: Gustavo Adolfo Chipantiza (2222022), Federico Gonzales Cárdenas (2225657), Juan Jacobo Delgado Melo (2230110), María de los Ángeles Valencia Hurtado (2235112). El código del reto es 2235112.

Mapa del notebook

#	Bloque	Qué produce
1	Configuración, datos y EDA	Pets (100 imágenes de benchmark) y PennFudanPed (170 imágenes, 423 peatones)
2	Fundamentos teóricos	los cuatro paradigmas y sus compromisos
3	Formulación matemática	ecuaciones y ejemplos numéricos
4	Implementación base	U-Net++, Mask R-CNN, YOLO26-seg y SAM funcionando
5	Transfer learning	Mask R-CNN reentrenado en PennFudanPed, medido con mask AP
6	Tres niveles	benchmark zero-shot, U-Net vs U-Net++, YOLO fine-tuning vs Mask R-CNN
7	Síntesis	tabla global y cuándo usar cuál

#	Bloque	Qué produce
8	Ejercicios	3 ejercicios con solución
9	Reto	pipeline YOLO → SAM, entrega `reto_s2_2235112.pt`

Conceptos desde cero

Todo el taller gira alrededor de una pregunta: ¿a qué objeto pertenece cada píxel de la imagen? Antes de mirar el código hay que entender estas piezas.

La imagen como números

Una foto en color es una matriz de alto × ancho × 3 (rojo, verde, azul). Cada valor va de 0 a 255. En PyTorch se reordena como `[3, H, W]` y se le llama **tensor**. Una foto de Pets de 500 × 333 píxeles son 499 500 números.

Tipos de segmentación

Tipo	Pregunta que responde	Ejemplo en el taller
Clasificación	¿qué hay en la foto?	"es un perro"
Detección	¿dónde está cada objeto? (una caja)	YOLO encierra al perro en un rectángulo
Segmentación semántica	¿qué clase tiene cada píxel?	U-Net: fondo, mascota, borde
Segmentación de instancias	¿a qué objeto concreto pertenece cada píxel?	Mask R-CNN: peatón 1, peatón 2
Segmentación por prompt	recorta lo que yo te señale	SAM: "recorta lo que está en esta caja"

Una **máscara** es una imagen del mismo tamaño que la foto, con 1 donde está el objeto y 0 donde no. Una **caja** (bounding box) se guarda como cuatro números en formato **XYXY**: `[x_min, y_min, x_max, y_max]`.

Métricas: cómo saber si una máscara es buena

IoU (Intersection over Union) compara la máscara predicha P con la verdadera G. Es el área donde coinciden dividida por el área total que cubren entre las dos. Va de 0 (no se tocan) a 1 (idénticas).

```
\mathrm{IoU} = \frac{|P \cap G|}{|P \cup G|}
```

Dice mide lo mismo pero premia más el solapamiento. Siempre es mayor o igual que el IoU de la misma máscara.

```
\mathrm{Dice} = \frac{2\,|P \cap G|}{|P| + |G|} = \frac{2\,\mathrm{IoU}}{1 + \mathrm{IoU}}
```

Ejemplo: si la mascota tiene 100 píxeles, la predicción 100 píxeles y coinciden 80, entonces IoU = 80 / 120 = 0.667 y Dice = 160 / 200 = 0.80.

mIoU es el promedio del IoU de cada clase (fondo, mascota, borde). **mask AP** (Average Precision, protocolo COCO) se usa en instancias: cuenta una detección como acierto si su IoU con un objeto real supera un umbral, y promedia la precisión sobre los umbrales 0.50, 0.55, … 0.95. **AP50** usa solo el umbral 0.5.

Latencia es el tiempo que tarda el modelo por imagen, en milisegundos (ms). 1000 ms / latencia = imágenes por segundo.

Entrenar, preentrenado y zero-shot

- **Entrenar** es ajustar los pesos del modelo para reducir una **pérdida** (loss), que mide qué tan lejos está la predicción de la verdad.
- **Preentrenado en COCO**: COCO es un dataset de 80 clases (persona, gato, perro, oso, oveja…). Los modelos ya vienen entrenados ahí.
- **Zero-shot**: usar el modelo en un dataset nuevo sin reentrenarlo. Funciona en Pets porque COCO ya tiene gatos y perros.
- **Fine-tuning / transfer learning**: partir de pesos preentrenados y seguir entrenando con datos nuevos (aquí, peatones de PennFudanPed).
- **Umbral de confianza (conf)**: el detector da a cada caja un puntaje de 0 a 1; solo se aceptan las que superan el umbral.

Hardware

La **GPU** (en Colab, CUDA) hace los cálculos en paralelo. La **VRAM** es su memoria; el notebook está pensado para 4 GB, por eso libera modelos con `free_memory` entre secciones (eso fue lo que borró a SAM durante el reto).

Sección 1: configuración y datos

Esta sección prepara todo lo que usan las demás: librerías, parámetros, funciones auxiliares y los dos datasets. Al terminar, el notebook tiene 100 imágenes de Pets para el benchmark y 170 imágenes de peatones divididas en train/val/test.

1.5 Instalación

La primera celda revisa si existen cuatro librerías y solo instala las que faltan.

```
REQUIRED = {"ultralytics": "ultralytics",               # YOLO26
            "segment_anything":
"git+https://github.com/facebookresearch/segment-anything.git",  # SAM
            "torchmetrics": "torchmetrics",             # mask AP
            "pycocotools": "pycocotools"}               # protocolo COCO
for module, pkg in REQUIRED.items():
    if importlib.util.find_spec(module) is None:
        subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",
pkg])
```

Salida: instaló `ultralytics`, `segment-anything` y `torchmetrics`; `pycocotools` ya venía en Colab.

La celda de imports carga numpy, matplotlib, PIL, OpenCV, PyTorch y torchvision. Imprimió `torch 2.11.0+cu130 | CUDA: True`: hay GPU disponible.

Configuración global

Parámetro	Valor	Para qué sirve
`IMG_SIZE`	128	tamaño al que se entrenan U-Net y U-Net++
`BATCH_SIZE`	16	imágenes por paso de entrenamiento

Parámetro	Valor	Para qué sirve
`SEED`	42	semilla: hace que los resultados se repitan
`MRCNN_MIN_SIZE`, `MAX_SIZE`	512, 768	resolución interna de Mask R-CNN, limitada por los 4 GB de VRAM
`BENCH_N`	100	imágenes de Pets del benchmark común
`SAM_CHECKPOINT`	`sam_vit_b_01ec64.pth`	pesos de SAM ViT-B (~375 MB)
`YOLO_WEIGHTS`	`yolo26n-seg.pt`	pesos de YOLO26 nano con segmentación (~7 MB)
`DEVICE`	cuda	dónde se calcula: GPU si existe
`USE_AMP`	True	precisión mixta: cálculos en 16 bits para ahorrar memoria y tiempo

Utilidades (celda larga de la Sesión 1)

No hay que memorizarla, pero sí saber qué hacen sus funciones principales:

- `set_seed`: fija la aleatoriedad de Python, numpy y PyTorch para que dos ejecuciones den lo mismo.
- `trimap_to_labels`: Pets anota cada píxel con un **trimap**: 1 = mascota, 2 = fondo, 3 = borde (la franja dudosa del contorno). La función lo pasa a 0 = fondo, 1 = mascota y, en modo binario, el borde = 255, que significa **ignorar** al calcular métricas.
- `build_pets_cache`: reduce Pets a 160 px y lo guarda en disco una sola vez. La imagen se reescala con interpolación bilineal (suave) y la máscara con **vecino más cercano**, porque promediar etiquetas inventaría clases que no existen (entre 1 y 2 saldría 1.5).
- `manifest_hash`: calcula una huella SHA-256 de la lista de imágenes de cada split. Si alguien cambia los datos, la huella cambia (versionado de datos).
- `DiceLoss`, `CELoss`, `ComboLoss`: funciones de pérdida para U-Net. ComboLoss suma entropía cruzada y Dice.
- `FundidoraPC`, `FundidoraUNet`: el encoder y la U-Net de la Sesión 1.
- `evaluate`, `fit`, `plot_history`: evaluar, entrenar y graficar curvas.

La celda siguiente añade `sync()` (espera a que la GPU termine, para medir tiempos reales) y `free_memory(*names)` (borra modelos de memoria). Luego carga Pets: **train = 3128, val = 552, test = 3669**, con `DATA_VERSION 356dbb55…`.

1.6 Benchmark de Pets a resolución original

Para comparar modelos que trabajan a tamaños distintos (U-Net a 128 px, Mask R-CNN a 512 px, SAM a 1024 px), todo se evalúa sobre la foto original. Se eligen 100 imágenes del test con la semilla fija 999, la misma del reto.

```
BENCH_SEED = 999
g = torch.Generator().manual_seed(BENCH_SEED)
bench_idx = torch.randperm(len(pets_test_raw), generator=g)[:100].tolist()
bench = [load_bench_item(i) for i in bench_idx]   # (imagen RGB, verdad
binaria, especie)

def binary_scores(pred, gt):
    valid = gt != IGNORE_INDEX                        # el borde no cuenta
    p, g_ = pred.astype(bool) & valid, (gt == 1) & valid
    inter, union = (p & g_).sum(), (p | g_).sum()
    return {"IoU": inter / union, "Dice": 2 * inter / (p.sum() + g_.sum())}

def gt_box(gt):                                       # caja "oráculo" a partir
de la verdad
    ys, xs = np.where((gt == 1) | (gt == IGNORE_INDEX))
    return np.array([xs.min(), ys.min(), xs.max(), ys.max()])
```

`binary_scores` es la implementación exacta del IoU y Dice explicados arriba, con el borde excluido. `gt_box` construye la caja perfecta a partir de la máscara verdadera; se llama **oráculo** porque usa información que en la vida real no tendríamos. Solo sirve para medir el techo de SAM, y está prohibido usarla en el reto.

Salida: `benchmark: 100 imágenes | tamaños ejemplo: [(333, 500), (333, 500), (355, 500)]`.

1.7 PennFudanPed

Es un dataset de peatones: cada píxel de la máscara guarda el **id de instancia** (0 = fondo, 1, 2, 3… = cada peatón). De ahí salen las máscaras individuales y las cajas sin anotar nada más.

```
inst = np.array(Image.open(.../"PedMasks"/f"{stem}_mask.png"))
ids = np.unique(inst)[1:]                                  # quitar el 0
(fondo)
masks = torch.from_numpy(inst[None] == ids[:, None, None])  # una máscara por
peatón
boxes = masks_to_boxes(masks)                              # caja que
encierra cada máscara
labels = torch.ones(len(ids))                              # una sola clase:
peatón
```

Los splits se hacen con semilla 42: 18 % test, 12 % val, el resto train.

Salida: `{'test': 31, 'val': 20, 'train': 119}` y `total instancias: 423 | media por imagen: 2.49`.

Cómo leer la figura: el primer panel es un histograma de peatones por imagen; la mayoría tiene 1 a 3 y pocas llegan a 7 u 8. Los otros tres paneles muestran fotos con cada peatón pintado de un color distinto y su caja en amarillo. Eso es exactamente segmentación de instancias: dos personas pegadas tienen dos colores distintos.

Por qué dos datasets: Pets tiene una sola mascota por foto y una máscara semántica, perfecta para comparar modelos en segmentación binaria. Pero no sirve para instancias; para eso está PennFudanPed.

Secciones 2 y 3: los cuatro modelos y sus fórmulas

Cada modelo responde de forma distinta a "¿qué es cada píxel?". Esta es la idea que hay que poder explicar de memoria.

Modelo	Idea en una frase	Analogía	Tipo de salida
U-Net++	encoder–decoder con muchas conexiones anidadas	un pintor que retoca el mismo lienzo a varias escalas	semántica
Mask R-CNN	primero cajas, luego una mini-red que recorta la silueta dentro de cada caja	encerrar a cada persona y luego recortarla	instancias
YOLO-seg	32 "máscaras base" compartidas que cada objeto mezcla con sus coeficientes	un kit de plantillas	instancias

Modelo	Idea en una frase	Analogía	Tipo de salida
SAM	segmenta lo que le señales con un punto o una caja, sin reentrenar	un asistente experto: "recorta esto"	máscara por prompt

U-Net++

La **U-Net** tiene dos mitades. El **encoder** reduce la imagen paso a paso (128 → 64 → 32 → 16 → 8 px) y aprende qué hay. El **decoder** la vuelve a agrandar para decir dónde está cada cosa. Las **skip connections** pasan detalles finos del encoder al decoder.

El problema, según Zhou et al. (2018), es la **brecha semántica**: el encoder trae bordes y texturas, el decoder trae significado, y mezclarlos de golpe no encaja bien. U-Net++ pone una rejilla de nodos intermedios que traducen poco a poco. Cada nodo X^{i,j} (nivel i, columna j) recibe todos los nodos anteriores de su nivel más el de abajo agrandado:

```
X^{i,j} = \mathcal{H}\Big(\big[X^{i,0}, X^{i,1}, \dots, X^{i,j-1},
\mathcal{U}(X^{i+1,j-1})\big]\Big), \quad j \ge 1,\ i + j \le 4
```

Aquí H son dos convoluciones 3×3 con BatchNorm y ReLU, U es agrandar ×2 y los corchetes significan concatenar canales. El decoder pasa de 4 nodos (U-Net) a 4 + 3 + 2 + 1 = 10 nodos.

Canales de entrada de un nodo, con anchos c = (16, 32, 64, 128, 256):

```
\mathrm{in}(X^{i,j}) = j \cdot c_i + c_{i+1}
```

Ejemplo: X^{0,3} recibe 3 × 16 + 32 = 80 canales. La celda de verificación lo confirmó: `X0,1: 48, X0,2: 64, X0,3: 80, X0,4: 96`.

Supervisión profunda (DS): cada nodo de la fila superior (X^{0,1} a X^{0,4}) produce una segmentación completa, y se entrena con el promedio de las cuatro pérdidas:

```
\mathcal{L}_{DS} = \frac{1}{4} \sum_{j=1}^{4} \mathcal{L}\big(\hat{Y}_j,
Y\big)
```

Poda: como cada salida ya es válida, en inferencia se puede cortar la red en el nivel L < 4 y ganar velocidad a cambio de algo de precisión.

Mask R-CNN

Parte de **Faster R-CNN** (detector de cajas en dos etapas: una RPN propone regiones y una cabeza las clasifica). Mask R-CNN (He et al., 2017) añade una tercera cabeza: una pequeña red que produce una máscara de **28 × 28 por clase** dentro de cada caja.

- **RoIAlign** recorta la región con interpolación bilineal, sin redondear coordenadas. Medio píxel de error casi no afecta una caja, pero destruye una máscara.
- **Máscaras desacopladas:** una sigmoide por clase en vez de un softmax; la clase la decide la otra cabeza, así que las clases no compiten por los píxeles.

```
\mathcal{L} = \mathcal{L}_{cls} + \mathcal{L}_{box} + \mathcal{L}_{mask},
\qquad \mathcal{L}_{mask} = -\frac{1}{m^2} \sum_{u,v} \Big[ t_{uv} \ln
\sigma(z^{(k)}_{uv}) + (1 - t_{uv}) \ln\big(1 - \sigma(z^{(k)}_{uv})\big)
\Big]
```

Con m = 28, t la máscara verdadera recortada a la caja y z^{(k)} el mapa de la clase verdadera k. Es una entropía cruzada binaria (BCE) promediada sobre los 784 píxeles. **Consecuencia:** todo objeto, grande o pequeño, se describe con solo 28 × 28 = 784 valores; eso pone un techo a la calidad del borde.

YOLO-seg

Segmentar con una red por caja es caro. YOLO-seg, siguiendo a YOLACT (Bolya et al., 2019), genera **k = 32 prototipos** (máscaras base para toda la imagen, a 1/4 de resolución) y, por cada objeto, un vector de 32 **coeficientes**. La máscara es la mezcla:

```
\mathbf{M}_n =
\operatorname{crop}_{\,b_n}\big(\sigma(\mathbf{P}\,\mathbf{c}_n)\big) > 0.5
```

σ es la sigmoide (convierte cualquier número en probabilidad entre 0 y 1) y crop recorta a la caja b_n. Cada máscara cuesta un producto matriz–vector, por eso YOLO-seg corre en tiempo real. YOLO26 además no usa NMS (supresión de no-máximos).

Ejemplo numérico del notebook, con 2 prototipos de 3 × 3 (P2 = −P1):

- c_A = (1, 0): σ(P1) da 0.881 en el bloque 2 × 2 superior izquierdo y 0.119 en el resto → 4 píxeles.
- c_B = (0, 1): la "L" complementaria → 5 píxeles.
- c = (1, 1): los prototipos se cancelan, σ(0) = 0.5 en todo → 0 píxeles. Máscara indefinida.

- c = (1, −0.5): el bloque sube a 0.953 → 4 píxeles más seguros. Los coeficientes negativos **restan** un prototipo, y así el modelo arma formas que ningún prototipo tiene por sí solo.

SAM (Segment Anything)

Kirillov et al. (2023) lo entrenaron con 1100 millones de máscaras para una tarea distinta: **dado un prompt (punto o caja), devolver una máscara válida**. No conoce clases, conoce objetos y partes.

```
\mathbf{E} = \mathrm{Enc}_{img}(\mathbf{x}_{1024}) \in \mathbb{R}^{256 \times
64 \times 64}, \qquad (\mathbf{M}_1, \mathbf{M}_2, \mathbf{M}_3,
\widehat{IoU}_{1..3}) = \mathrm{Dec}(\mathbf{E}, \mathbf{Q})
```

1. **Encoder de imagen** (ViT, pesado): corre una vez por imagen, escalada a 1024 × 1024, y da un embedding E. Es lo lento (~93.7 M parámetros, 89.7 M solo en el encoder).
2. **Encoder de prompts** (ligero): convierte puntos y cajas en vectores Q.
3. **Decoder de máscaras** (ligero): cruza E y Q y devuelve **3 máscaras** con un IoU predicho para cada una.

¿Por qué 3? **Ambigüedad:** un punto sobre la camisa puede ser "la camisa", "el torso" o "la persona". **Limitación clave:** SAM no sabe *qué* es algo; hay que decirle *dónde* mirar. Por eso el reto lo combina con un detector.

Sección 4: los cuatro modelos funcionando

Aquí se construye U-Net++ y se cargan Mask R-CNN, YOLO26-seg y SAM con pesos ya entrenados. El hallazgo más importante para el reto aparece al final: SAM con una caja que cubre casi toda la foto se equivoca (IoU 0.80), aunque con un punto bien puesto llega a 0.98.

4.1 U-Net++ sobre FundidoraPC

La clase `FundidoraUNetPP` implementa la fórmula de los nodos X^{i,j}. Usa el mismo encoder que la U-Net de la Sesión 1 para que la comparación sea justa.

```
for j in range(1, 5):
    for i in range(0, 5 - j):
        in_c = j * self.ch[i] + self.ch[i + 1]           # j*c_i + c_{i+1}
        self.nodes[f"x{i}_{j}"] = nn.Sequential(conv_bn_relu(in_c,
self.ch[i]),
                                                conv_bn_relu(self.ch[i],
self.ch[i]))

# en forward: cada nodo concatena sus hermanos de nivel + el de abajo
agrandado
up = F.interpolate(X[(i + 1, j - 1)], size=X[(i, 0)].shape[-2:],
mode="bilinear")
X[(i, j)] = self.nodes[f"x{i}_{j}"](torch.cat([X[(i, k)] for k in range(j)] +
[up], dim=1))
```

- `prune_level=L` calcula solo los nodos con i + j ⩽ L (poda).
- En entrenamiento con supervisión profunda devuelve 4 salidas; `DeepSupervisionLoss` promedia su pérdida.
- `Pruned(model, level)` es un envoltorio para evaluar la red podada.

Salida: en entrenamiento, 4 salidas de forma `(2, 3, 128, 128)` (2 imágenes, 3 clases, 128 × 128 px); podada a L = 1, 2, 3, 4 siempre da `(2, 3, 128, 128)`. **Parámetros: U-Net++ 1 501 756 frente a U-Net 1 742 547.** U-Net++ tiene más nodos pero menos parámetros, porque sus nodos son más delgados (trabajan con los canales c_i del nivel en vez de los anchos del decoder de U-Net).

4.2 Mask R-CNN preentrenado en COCO

```
model =
maskrcnn_resnet50_fpn_v2(weights=MaskRCNN_ResNet50_FPN_V2_Weights.DEFAULT,
                                 min_size=512, max_size=768)
if num_classes != 91:          # transfer learning: se cambian las cabezas
    model.roi_heads.box_predictor = FastRCNNPredictor(in_f, num_classes)
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_m, 256,
num_classes)
```

`build_maskrcnn` sirve para los dos usos: con 91 clases es el modelo de COCO tal cual; con otro número reemplaza las cabezas de clasificación y máscara (se usa en la sección 5 con 2 clases: fondo y peatón). `mrcnn_predict` pasa la imagen en [0, 1] y descarta detecciones con score < 0.5.

Salida: `cat = 17, dog = 18, parámetros: 46 359 409`. La cabeza de máscara termina en `Conv2d(256, 91, 1×1)`: 91 mapas, uno por clase (el desacoplamiento). El RoIAlign recorta 14 × 14, y una convolución transpuesta lo sube a 28 × 28.

Figura (gato blanco): a la izquierda, la caja amarilla `cat 1.00`; al centro, el mapa de probabilidad de la máscara (amarillo = alta); a la derecha, la verdad: naranja mascota, azul borde ignorado. La máscara sigue bien la silueta pero es suave en los bordes, consecuencia de la resolución 28 × 28. Detección: `('cat', 0.997)`.

El techo de 28 × 28. La celda toma la máscara verdadera, la reduce como lo haría Mask R-CNN, la vuelve a pegar y mide su IoU. Es el máximo alcanzable aunque la red fuera perfecta:

Resolución de la máscara	IoU techo medio (50 imágenes)
14 × 14	0.9548
28 × 28	0.9779
56 × 56	0.9861
112 × 112	0.9886

Interpretación: con 28 × 28 se pierde un 2.2 % de IoU solo por la resolución. Duplicar a 56 recupera 0.8 puntos, y después casi nada. Las pérdidas están en orejas, colas y patas delgadas.

4.3 YOLO26-seg y sus prototipos

```
yolo = YOLO("yolo26n-seg.pt")
YOLO_PET_CLASSES = [k for k, v in yolo.names.items() if v in ("cat", "dog")]
# [15, 16]

def to_bgr(img_rgb):                    # Ultralytics asume BGR (OpenCV)
    return np.ascontiguousarray(img_rgb[..., ::-1])

hook = head.proto.register_forward_hook(lambda mod, inp, out:
captured.__setitem__("proto", out))
res = yolo.predict(to_bgr(img), retina_masks=True, conf=0.25)[0]
```

- **BGR vs RGB:** OpenCV guarda los colores al revés (azul, verde, rojo). Si se le pasa RGB a YOLO, ve un gato azul. `[..., ::-1]` invierte el orden de los canales.

- `retina_masks=True` devuelve las máscaras al tamaño original.
- Un **forward hook** es un "espía" que guarda la salida de una capa mientras la red corre; aquí captura los prototipos.

Salida: `cabeza: Segment26 | prototipos k = 32 | clases: 80 | cat = [15] | dog = [16]`; detección `('cat', 0.889)`. Ojo: en YOLO gato y perro son 15 y 16, no 17 y 18 como en Mask R-CNN; cada librería numera COCO distinto.

Figura de prototipos (32 mapas de 112 × 160): rojo = valores positivos, azul = negativos. No son partes con nombre: el 8, 10 y 17 se encienden sobre el gato; el 5, 6 y 13 marcan el fondo (casi su negativo); el 7 y el 19 resaltan contornos; el 11 y el 15 tienen franjas verticales, que dan información de **posición**. La máscara final (arriba, en blanco) es una suma con signo de todos ellos.

4.4 SAM: punto, caja y ambigüedad

```
sam = sam_model_registry["vit_b"]
(checkpoint=SAM_CHECKPOINT).to(DEVICE).eval()
predictor = SamPredictor(sam)
predictor.set_image(img)                # encoder ViT: una vez por imagen
(caro)

# punto: el píxel de la mascota más lejos de cualquier borde
dist = cv2.distanceTransform(fg, cv2.DIST_L2, 5)
py, px = np.unravel_index(dist.argmax(), dist.shape)
masks_pt, scores_pt, _ = predictor.predict(point_coords=np.array([[px, py]]),
                                           point_labels=np.array([1]),
multimask_output=True)
masks_bx, _, _ = predictor.predict(box=gt_box(gt), multimask_output=False)
# caja oráculo
```

La **transformada de distancia** asigna a cada píxel de la mascota su distancia al borde; el máximo es el punto "más adentro". `point_labels=1` indica punto positivo ("esto es objeto").

Salida: `SAM ViT-B: 93 735 472 parámetros | encoder: 89 670 912` (el 96 % del modelo es el encoder). Tiempos: `encoder 584 ms | decoder 233.3 ms`. El decoder mide alto porque es la primera llamada (calentamiento de la GPU); normalmente son unos pocos ms.

Figura (cinco paneles):

Prompt	IoU predicho por SAM	IoU real
punto → máscara 0 (gato completo)	0.90	0.98
punto → máscara 1 (cabeza y pata)	0.80	0.72
punto → máscara 2 (solo el ojo)	0.97	0.01
caja oráculo	—	0.80

Interpretación: el punto cayó sobre el ojo, y SAM propuso tres lecturas: el gato, una parte y el ojo. La más confiada (0.97) fue la peor (0.01): **el IoU predicho mide si la máscara es un objeto bien formado, no si es el objeto que querías**. La caja cubría casi toda la foto, y SAM metió la manta negra de abajo, bajando a 0.80. Esto es exactamente lo que después hace perder al pipeline del reto en primeros planos.

Calibración (gráfico de dispersión): cada punto es una de las 3 máscaras de 30 imágenes. Eje x = IoU predicho, eje y = IoU real; la diagonal sería calibración perfecta. Hay muchos puntos con predicho > 0.9 y real < 0.4. Correlación = **0.541**: moderada. No se puede confiar ciegamente en el puntaje de SAM.

Generador automático: pone una rejilla de 16 × 16 puntos y filtra por IoU predicho ⩾ 0.86 y estabilidad ⩾ 0.9. Encontró **13 máscaras en 1.7 s** en la foto del yorkshire: perro, cojín, sábana, nariz, orejas… pero sin saber cuál es el perro. La celda termina con `free_memory("amg", "auto_masks", "predictor", "sam")`.

Sección 5: transfer learning de Mask R-CNN a peatones

Mask R-CNN, preentrenado en COCO con 91 clases, se adaptó a 2 clases (fondo y peatón) en 8 épocas de unos 21 s cada una. En el test de PennFudanPed logró **mask AP 0.829, AP50 0.994 y AP75 0.962**.

Los tres pasos

1. Cargar Mask R-CNN con pesos de COCO.
2. **Reemplazar** las dos cabezas que dependen del número de clases: `box_predictor` (91 → 2) y `mask_predictor` (91 → 2 mapas).
3. Entrenar el resto con una tasa de aprendizaje pequeña, para no destruir lo que ya sabe.

BatchNorm congelado: con lotes de solo 2 imágenes, las estadísticas de BatchNorm serían ruido. torchvision lo resuelve usando `FrozenBatchNorm2d` (estadísticas fijas de COCO) y congelando las primeras capas del backbone.

Código clave

```
pf_train_tf = v2.Compose([v2.RandomHorizontalFlip(0.5),          # espejo:
imagen, cajas y máscaras a la vez
                          v2.RandomPhotometricDistort(p=0.5),     # cambios
de brillo/color
                          v2.ToDtype(torch.float32, scale=True),  # 0-255 ->
0-1
                          v2.SanitizeBoundingBoxes()])            # elimina
cajas degeneradas

mr_pf = build_maskrcnn(num_classes=2)
opt = torch.optim.SGD(trainable, lr=0.005, momentum=0.9, weight_decay=5e-4)
sch = torch.optim.lr_scheduler.OneCycleLR(opt, max_lr=0.005, total_steps=...,
pct_start=0.15)

for imgs, tgts in pf_train_loader:
    with torch.autocast(DEVICE.type, enabled=USE_AMP):
        losses = mr_pf(imgs, tgts)     # classifier, box_reg, mask,
objectness, rpn_box_reg
        loss = sum(losses.values())
    scaler.scale(loss).backward(); scaler.step(opt); scaler.update();
sch.step()
```

- **Aumentación de datos:** variar las imágenes al entrenar para que el modelo no memorice. `tv_tensors` hace que voltear la imagen voltee también sus cajas y máscaras.
- **SGD con momento:** el optimizador clásico; `weight_decay` penaliza pesos grandes (regularización).
- **OneCycleLR:** la tasa de aprendizaje sube durante el primer 15 % y luego baja suave hasta casi 0.
- **autocast + GradScaler:** precisión mixta (16 bits) para ahorrar memoria sin perder estabilidad.
- `evaluate_instances` mide box AP y mask AP con el protocolo COCO (torchmetrics).

Salida inicial: nuevas cabezas `Linear(1024 → 2)` y `Conv2d(256 → 2, 1×1)`; entrenables **45 655 067 de 45 880 411** parámetros (el resto está congelado).

Las cinco pérdidas

Pérdida	Qué castiga	Época 1	Época 8
classifier	clasificar mal una región (peatón o fondo)	0.160	0.019
box_reg	caja final mal ajustada	0.227	0.025
mask	máscara 28 × 28 distinta de la verdad (la BCE de la sección 3)	0.400	0.111
objectness	la RPN no distingue objeto de fondo	0.021	0.001
rpn_box_reg	las propuestas de la RPN mal ubicadas	0.004	0.001

Cómo leer las curvas

Izquierda (pérdidas de entrenamiento): todas caen fuerte en la primera época y luego se aplanan. La de máscara (verde) es la más alta y la que más baja (0.40 → 0.11): es lo único realmente nuevo, porque detectar personas ya lo sabía de COCO.

Derecha (validación): mask AP50 (naranja) se mantiene en ~0.98 desde la época 1: el modelo **encuentra** a los peatones casi desde el principio. Mask AP (azul) sube de 0.704 a 0.787 y box AP (verde) de ~0.74 a ~0.84: lo que mejora con el entrenamiento es **la precisión del contorno**, no la detección. La caída de la época 6 (AP50 0.965) es ruido normal con solo 20 imágenes de validación.

Resultado en test

Métrica	Valor	Significado
bbox AP	0.8962	cajas, promedio de IoU 0.50 a 0.95
bbox AP50	0.9939	casi todos los peatones encontrados
mask AP	**0.829**	máscaras, promedio de IoU 0.50 a 0.95
mask AP50	0.9939	máscaras aceptables casi siempre
mask AP75	0.9623	máscaras muy ajustadas en el 96 %

La brecha entre AP50 y AP75 es pequeña (0.994 → 0.962): los contornos son buenos. El AP promedio (0.829) baja porque a IoU 0.90–0.95 el techo de 28 × 28 empieza a pesar.

Figura de ejemplo: foto de test con 2 peatones reales; Mask R-CNN encontró **2 instancias** y pintó a cada una de un color, incluida la mochila de la mujer de la izquierda. Esa es la ventaja de instancias: dos personas, dos máscaras.

Sección 6: tres niveles de complejidad

Los tres experimentos dan tres conclusiones: sin entrenar, Mask R-CNN es el más preciso en Pets (IoU 0.9719) y YOLO el más rápido (18 ms); U-Net++ gana a U-Net por solo 0.01 de mIoU; y YOLO26n-seg reentrenado queda a 1.9 puntos de mask AP de Mask R-CNN siendo 6.7 veces más rápido.

6.1 Nivel fácil: benchmark zero-shot en Pets

Ningún modelo se entrena. Cada uno debe decir "mascota sí/no" en las 100 fotos del benchmark. La función central, que el reto reutiliza, es `run_benchmark`:

```
def run_benchmark(name, predict_mask_fn):
    for img, gt, _ in bench:
        sync(); t0 = time.time()
        pm = predict_mask_fn(img)                # máscara binaria (H, W)
        sync(); times.append(time.time() - t0)
        empty += int(pm.sum() == 0)              # ¿no encontró nada?
        scores.append(binary_scores(pm, gt))
    return {"IoU": media, "Dice": media, "sin_deteccion": empty / 100,
            "ms_por_imagen": 1000 * np.median(times), "per_image_IoU": [...]}

def yolo_pet_mask(img, conf=0.25):             # la referencia del reto
    r = yolo.predict(to_bgr(img), retina_masks=True, conf=conf,
classes=YOLO_PET_CLASSES)[0]
    return (r.masks.data > 0.5).any(0) if r.masks is not None else
np.zeros(...)
```

Cada modelo une las máscaras de sus detecciones de gato o perro (`.any(0)` = unión). SAM recibe la caja oráculo, así que **no es una comparación justa**: mide su techo cuando alguien le dice dónde mirar. La latencia usa la **mediana**, que no se deja afectar por una imagen lenta.

Salida:

Modelo	IoU	Dice	Sin detección	ms/img
Mask R-CNN (COCO)	**0.9719**	**0.9842**	0 %	119
YOLO26n-seg (COCO)	0.9419	0.9541	3 %	**18**
SAM ViT-B + caja oráculo	0.8786	0.8918	0 %	402

Cómo leer las tres gráficas:

- **Barras de IoU:** las tres están entre 0.88 y 0.97; a esta escala parecen casi iguales.
- **Barras de latencia:** aquí la diferencia es enorme: YOLO 18 ms, Mask R-CNN 119 ms, SAM 402 ms (casi todo es el encoder).
- **Diagrama de caja (boxplot) del IoU por imagen:** la caja es donde está la mitad central de las imágenes; los círculos son imágenes atípicas. Las tres cajas están pegadas a 1.0: en la mayoría de fotos los tres modelos son casi perfectos. Lo que baja la media son pocos casos: YOLO tiene **3 círculos en 0.0** (las 3 fotos donde no detectó nada) y SAM tiene varios ceros, aunque su caja era la correcta.

Lección clave: el IoU medio de YOLO (0.9419) no es bajo porque sus máscaras sean malas, sino porque en 3 fotos no encontró a la mascota. Hay dos fallas distintas: **no encontrar** (detector) y **recortar mal** (máscara). Esa idea es la base del reto.

Figura de 3 fotos × 4 columnas: en el gato blanco, Mask R-CNN da 1.00, YOLO 0.99 y SAM oráculo solo 0.80, porque la caja ocupa toda la foto y SAM mete la manta. En el yorkshire y el gato sentado los tres pasan de 0.96.

6.2 Nivel intermedio: U-Net vs U-Net++ y poda

Tres modelos con el mismo encoder, los mismos datos (3 clases), la misma pérdida (0.5 CE + 0.5 Dice), AdamW con OneCycle y 12 épocas:

```
base_loss = ComboLoss([CELoss(), DiceLoss()], [0.5, 0.5])
unet, h_unet = train_config(FundidoraUNet(3), base_loss, 12)
uppn, h_uppn = train_config(FundidoraUNetPP(3, deep_supervision=False),
base_loss, 12)
uppd, h_uppd = train_config(FundidoraUNetPP(3, deep_supervision=True),
DeepSupervisionLoss(base_loss), 12)
```

Curvas de entrenamiento: a la izquierda, pérdida (punteada = train, continua = val); a la derecha, mIoU de validación por época. Los tres terminan cerca de 0.70. La U-Net tiene un pico de pérdida de validación en la época 2 y su mIoU cae a ~0.37 ese mismo paso; es inestabilidad del inicio con la tasa de aprendizaje alta, y se recupera. La pérdida de train de U-Net++ DS (morada) queda mucho más arriba (0.437), pero no es comparable: promedia también las salidas superficiales, que son peores.

Resultado en test:

Modelo	Parámetros	ms/img	mIoU	IoU mascota	IoU borde
U-Net	1 742 547	3.75	0.7085	0.7738	0.4773
U-Net++	1 501 756	5.40	**0.7184**	**0.7827**	**0.4961**
U-Net++ DS (L = 4)	1 501 756	5.59	0.7147	0.7791	0.4891
U-Net++ DS (L = 3)	1 501 756	3.50	0.6746	0.7316	0.4527
U-Net++ DS (L = 2)	1 501 756	2.06	0.5874	0.6341	0.3643
U-Net++ DS (L = 1)	1 501 756	0.82	0.4578	0.4812	0.2387

Gráfica de compromiso precisión–velocidad: eje x = latencia, eje y = mIoU. La línea azul es la U-Net++ podada: cada nivel menos pierde bastante precisión. El cuadrado azul (U-Net) queda **por encima** de L = 3 con casi la misma latencia: la U-Net completa domina a la red podada.

Respuestas del grupo:

1. **¿U-Net++ supera a U-Net?** Sí, por poco: 0.7184 vs 0.7085 (+0.010). La mayor ganancia es en el **borde** (+0.019), lo que coincide con la idea del paper: las skips anidadas ayudan en detalles finos. Pero es una sola semilla y 0.01 puede ser variación aleatoria. Cuesta +44 % de latencia y +48 % de tiempo por época, aunque tenga menos parámetros. **Lo que crece no es el tamaño del modelo sino la memoria de activaciones**: 10 nodos en vez de 4, muchos a alta resolución.
2. **¿La supervisión profunda ayuda?** No a la salida final (0.7147 con DS vs 0.7184 sin DS). Su función es **entrenar las salidas intermedias** para que la poda sea posible.
3. **¿Cuánto cuesta podar?** L = 3 ahorra el 37 % del tiempo pero pierde 0.040 de mIoU; L = 1 pierde 0.257. Con este encoder la poda no conviene: la U-Net completa es casi igual de rápida (3.75 ms) y mejor.

6.3 Nivel difícil: YOLO26n-seg reentrenado vs Mask R-CNN

YOLO no lee máscaras: necesita **polígonos normalizados**, una línea por objeto con el formato `clase x1 y1 x2 y2 … xn yn` y coordenadas entre 0 y 1.

```
def mask_to_polygon(mask, eps_frac=0.002):
    contours, _ = cv2.findContours(mask.astype(np.uint8), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_NONE)
    c = max(contours, key=cv2.contourArea)                                  # el
contorno más grande
    c = cv2.approxPolyDP(c, eps_frac * cv2.arcLength(c, True), True)      #
simplificar (Douglas-Peucker)
    return c[:, 0, :]

# por cada peatón:  "0 x1/W y1/H x2/W y2/H ..."
```

Control de calidad: se vuelve a pintar el polígono y se compara con la máscara original. **IoU medio 0.9810 (mínimo 0.9289)**: convertir a polígono pierde un ~2 %, sobre todo en personas con huecos (entre brazos y cuerpo), porque solo se guarda el contorno externo. Exportados: 277 polígonos en train, 61 en val, 85 en test.

Entrenamiento: 30 épocas, imágenes de 512 px, batch 8, AdamW con lr 0.001 y coseno, `close_mosaic=5` (las últimas 5 épocas sin mosaico, para ver imágenes normales), `patience=15` (parada temprana). Tardó **190 s** en una Tesla T4. La validación de Ultralytics del mejor checkpoint dio Mask mAP50 0.978 y mAP50-95 0.746.

Comparación con el mismo evaluador y el mismo test:

Modelo	Parámetros	ms/img	box AP	mask AP	mask AP50	mask AP75
Mask R-CNN R50-FPN v2	45 880 411	122.1	**0.8962**	**0.8290**	**0.9939**	**0.9623**
YOLO26n-seg	3 053 055	**18.2**	0.8350	0.8102	0.9757	0.9427

Respuestas del grupo:

- **(a) AP vs latencia:** YOLO es **6.7× más rápido** y pierde solo **1.9 puntos** de mask AP. Pierde más en box AP (−6.1 puntos): falla más ubicando cajas que dibujando máscaras.
- **(b) Finura del contorno:** la caída de AP50 a AP75 es casi igual (−3.2 Mask R-CNN, −3.3 YOLO), así que los contornos son comparables. La desventaja de YOLO está en **encontrar** algunos peatones. Matices: YOLO se entrenó con polígonos (techo ~0.98) y el test es pequeño (31 imágenes).
- **(c) Tamaño:** YOLO tiene 3.05 M parámetros, **~1/15** de Mask R-CNN.

Conclusión: para contar peatones en tiempo real, YOLO26n-seg; Mask R-CNN solo si importa más encontrar cada instancia que la velocidad.

Sección 7: comparación final y cuándo usar cada modelo

En el benchmark de Pets ganan los modelos preentrenados en COCO, aunque nunca vieron Pets: Mask R-CNN logra IoU 0.9719, mientras la U-Net++ entrenada en Pets se queda en 0.8060.

Para meter la U-Net y la U-Net++ al mismo benchmark, la celda reduce la foto a 128 px, predice y vuelve a agrandar el resultado. Decide mascota cuando el logit de mascota supera al de fondo, y descarta el canal "borde":

```
def semantic_pet_mask(model, img):
    x = normalize(img / 255, MEAN, STD)                          # misma
normalización del entrenamiento
    x = F.interpolate(x, size=(128, 128), mode="bilinear")
    logits = F.interpolate(model(x), size=img.shape[:2], mode="bilinear")[0]
    return (logits[1] > logits[0])                               # mascota >
fondo
```

Modelo	IoU	Dice	Sin detección	ms/img	¿Entrenado en Pets?
Mask R-CNN (COCO)	**0.9719**	**0.9842**	0 %	119	no
YOLO26n-seg (COCO)	0.9419	0.9541	3 %	18	no
SAM ViT-B + caja oráculo	0.8786	0.8918	0 %	402	no
FundidoraUNet++ DS (Pets 128)	0.8060	0.8715	0 %	12	sí
FundidoraUNet (Pets 128)	0.7886	0.8580	1 %	**8**	sí

Por qué ganan los que no vieron Pets: Mask R-CNN y YOLO se entrenaron con miles de gatos y perros de COCO, a alta resolución y con redes mucho más grandes. Las U-Net del taller son pequeñas (1.5–1.7 M parámetros) y trabajan a 128 px, así que al agrandar a la foto original pierden detalle en los bordes. U-Net++ supera a U-Net también aquí (+0.017), en línea con la sección 6.2.

Cuándo usar cuál

Situación	Elección	Por qué
Clases fijas, datos etiquetados, bordes finos (médico, industrial)	U-Net / U-Net++	resolución completa, entrenamiento simple
Objetos que se solapan y hay que contar	Mask R-CNN	una máscara por instancia gracias a las cajas
Tiempo real (cámaras, móviles)	YOLO-seg	cada objeto extra casi no cuesta
Pocas o ninguna etiqueta, pero alguien señala el objeto	SAM + detector, clic o texto	generaliza sin reentrenar
Anotar un dataset nuevo rápido	SAM asistido + corrección humana	reduce el tiempo por máscara

Errores frecuentes (útiles si el profe pregunta "¿qué pasaría si…?")

Síntoma	Causa	Solución
YOLO rinde menos de lo esperado	se le pasó RGB y él asume BGR	`img[..., ::-1]` (`to_bgr`)
Máscaras de YOLO desplazadas o pequeñas	faltó `retina_masks=True`	activarlo
Mask R-CNN peor tras normalizar	doble normalización	pasar la imagen en [0, 1]; el modelo normaliza solo
SAM recorta una parte (ojo, collar)	punto ambiguo	`multimask_output=True` o usar caja
SAM lentísimo	`set_image` por cada prompt	un `set_image` por imagen, muchos `predict`
`CUDA out of memory`	varios modelos grandes cargados	cargar, evaluar y liberar con `free_memory`
U-Net++ DS: `list has no attribute float`	la pérdida recibe la lista de salidas	envolver con `DeepSupervisionLoss`

Sección 8: ejercicios

Los tres ejercicios confirman la teoría con código: U-Net++ de profundidad 3 tiene 6 nodos, dos prototipos opuestos no pueden dibujar una sola columna, y SAM con 3 puntos + 1 negativo supera a la caja oráculo en media (0.9831 vs 0.7878) porque esa caja estaba mal construida en 2 fotos.

Ejercicio 1: U-Net++ de profundidad 3

Pregunta: ¿cuántos nodos y cuántos canales de entrada tiene una U-Net++ de profundidad 3 con anchos (16, 32, 64, 128)?

Cálculo: nodos con $j \geq 1$ e $i + j \leq 3$ → $3 + 2 + 1 =$ **6** (frente a 10 con profundidad 4). Canales con $j \cdot c_i + c_{i+1}$:

Nodo	Cálculo	Canales
X^{0,1}	1·16 + 32	48
X^{1,1}	1·32 + 64	96
X^{2,1}	1·64 + 128	192
X^{0,2}	2·16 + 32	64
X^{1,2}	2·32 + 64	128
X^{0,3}	3·16 + 32	80

Código: la clase original tenía la profundidad fija (`range(1, 5)`) y con 3 bloques fallaría con `IndexError` al buscar `self.ch[4]` . Por eso se creó la subclase `FundidoraUNetPPDepth` con `self.D = len(self.backbone.widths)` . Un `assert real == teo` comprueba que los canales del modelo coinciden con la teoría.

Salida: 6 nodos, mismos canales, 3 salidas de supervisión profunda de `(2, 3, 128, 128)` . **Parámetros: 359 753 (profundidad 3) frente a 1 501 756 (profundidad 4).** Quitar el nivel más profundo, el de 256 canales, elimina el 76 % de los parámetros.

Ejercicio 2: una máscara de solo la columna derecha

Pregunta: con P1 y P2 (y un sesgo), ¿se puede obtener una máscara de solo la columna derecha?

Respuesta: no. Como P2 = −P1, toda combinación es (c1 − c2) · P1 + b. P1 solo tiene dos valores (+2 en la esquina, −2 en la "L"), así que todos los píxeles de la L reciben el mismo logit: la columna derecha y la fila de abajo no se pueden separar. Las únicas máscaras posibles son vacía, esquina, L o todo. Una búsqueda en rejilla lo confirmó: **el mejor IoU posible es 0.600** (la L completa acierta 3 de 5 píxeles).

Solución: añadir P3 = +2 en la columna derecha y −2 en el resto.

- c = (0, 0, 1) → σ(2) = 0.881 en la columna y 0.119 fuera: exacto.
- c = (0, 0.5, 1) → 0.953 en la columna, 0.269 y 0.047 en el resto: también exacto. P2 empuja la L, P3 empuja la columna, y solo la columna pasa de 0.5.

Moraleja: con 32 prototipos variados, YOLO arma formas que ningún prototipo tiene solo. Con prototipos redundantes (uno el negativo del otro) no se gana expresividad.

Ejercicio 3: SAM con puntos vs caja

Construcción de los prompts (ambos usan la verdad, así que no son un pipeline real):

- **3 puntos positivos:** se divide la mascota en 3 franjas a lo largo de su lado más largo y en cada una se toma el píxel más alejado del borde. Quedan repartidos: cabeza, tronco y patas.
- **1 punto negativo:** el píxel de fondo más alejado de la mascota dentro de su caja (fondo "confundible").

```
predictor.set_image(img)                        # un encoder, dos prompts
m_pts, _, _ = predictor.predict(point_coords=coords, point_labels=labels)   #
labels = [1, 1, 1, 0]
m_box, _, _ = predictor.predict(box=gt_box(gt))
```

Resultado en 20 imágenes:

Prompt	IoU medio	Dice medio	IoU mediano
3 pos + 1 neg	**0.9831**	**0.9912**	0.9940
Caja oráculo	0.7878	0.7941	**0.9950**

Δ IoU = **+0.1952** a favor de los puntos, pero ganan solo en **7 de 20**.

Interpretación: la mediana dice que en la foto típica los dos son iguales (0.994 vs 0.995). La diferencia de medias la producen **dos fallos totales de la caja**.

Figura (4 filas: las 2 peores y las 2 mejores para los puntos; estrella verde = punto positivo, X roja = negativo):

- **Fila 1, chihuahua (Δ −0.12):** con puntos, SAM deja fuera las orejas (no tenían punto encima) y se extiende al cojín: 0.87. La caja fija la extensión completa: 0.99.
- **Fila 2, perro que llena la foto (Δ −0.02):** los puntos dejan un hueco en el hocico: 0.97 vs 0.99.
- **Fila 3, perro en el sofá (Δ +0.99):** la caja oráculo llega hasta el borde derecho de la foto, y SAM segmenta el sofá: 0.00. Los puntos: 0.99.
- **Fila 4, foto circular (Δ +0.99):** la caja es [0, 0, 493, 499], toda la imagen, y SAM segmenta las esquinas negras: 0.00. Los puntos: 0.99.

La verificación compara la caja de solo mascota con la caja oráculo. `gt_box` incluye los píxeles de borde del trimap (255). Normalmente eso solo agranda la caja 15–25 px por lado (1.3–1.7×) y no afecta. En la **imagen 8** la mascota termina en x = 357 pero la caja llega a x = 499 (2.0×), extendiéndose solo hacia la derecha. En la **imagen 11** la caja es 2.3× la mascota. Esos dos casos son los dos ceros.

Conclusión: con prompts correctos, caja y puntos son casi equivalentes; la caja gana en extremidades finas y los puntos en huecos interiores. Consecuencia para el notebook: el 0.879 de "SAM + caja oráculo" en el benchmark 6.1 **está subestimado** por esa caja inflada. Lo ideal sería combinar caja + puntos en el mismo `predict`.

Sección 9: el reto Detectar → Refinar

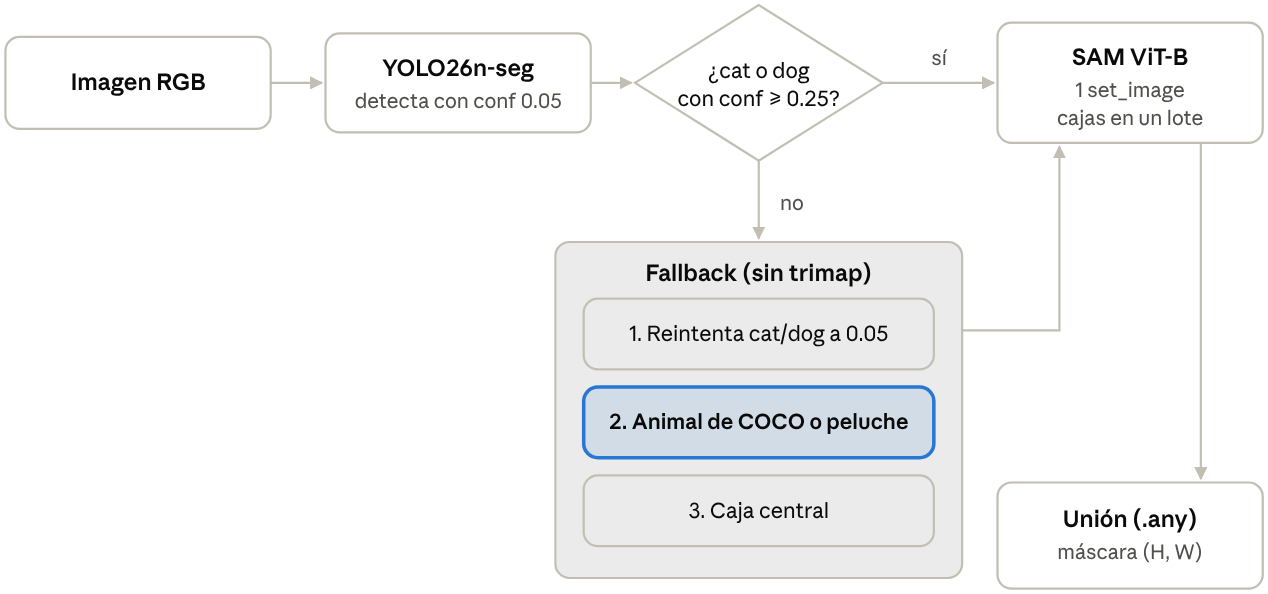
El pipeline YOLO → SAM superó a YOLO solo: **IoU 0.9523 frente a 0.9419 (+0.0104)** y Dice 0.9709 frente a 0.9541, a cambio de pasar de 19 a 469 ms por imagen. El hallazgo central es que casi toda la ganancia viene del fallback, que rescató 3 mascotas que YOLO había clasificado como oveja u oso.

La idea

La sección 6.1 mostró que YOLO **encuentra** bien a las mascotas y que SAM **recorta** bien cuando le dicen dónde mirar. El reto los encadena:

```
\text{imagen} \xrightarrow{\text{YOLO26n-seg}} \text{cajas cat/dog}
\xrightarrow{\text{SAM ViT-B}} \text{máscaras} \xrightarrow{\ \cup\ }
\text{máscara de mascota}
```

El fallback rescató las 3 fotos que YOLO clasificó mal



Se activó en 3 imágenes; las 3 en el paso 2 (sheep, bear, bear)

pipeline del reto · YOLO, decisión, fallback de 3 pasos, SAM

Si YOLO encuentra un gato o perro con confianza ≥ 0.25, sus cajas van directo a SAM; si no, el fallback busca una caja en tres pasos y también la manda a SAM.

Reglas: criterio `IoU_pipeline > IoU_YOLO + 0.0`, las mismas 100 imágenes (semilla 999), prohibido usar el trimap dentro del pipeline y prohibido ajustar umbrales mirando el IoU imagen por imagen.

Celda de parámetros

`STUDENT_CODE = "2235112"` y `TARGET_GAIN = 0.0`. El `assert BENCH_SEED == 999` garantiza que se evalúa sobre las mismas fotos del benchmark. Salida: `imágenes de evaluación: 100`.

Parte A: nuestro código

0. Recargar SAM si hace falta. El notebook libera SAM en 4.4 y 6.1 para ahorrar VRAM, y la primera vez la evaluación falló con `NameError: predictor`. Por eso la celda empieza así:

```
if "predictor" not in globals():          # SAM se liberó antes -> recargarlo
    sam = sam_model_registry["vit_b"]
(checkpoint=SAM_CHECKPOINT).to(DEVICE).eval()
    predictor = SamPredictor(sam)
```

1. `yolo_pet_boxes`: quedarse con las cajas de gato o perro.

```
def yolo_pet_boxes(result, conf_thres=0.25):
    b = result.boxes
    if b is None or len(b) == 0:
        return np.zeros((0, 4), np.float32)                    # sin
detecciones
    cls  = b.cls.cpu().numpy().astype(int)                      # clase de
cada caja
    conf = b.conf.cpu().numpy()                                 # confianza
de cada caja
    keep = np.isin(cls, YOLO_PET_CLASSES) & (conf >= conf_thres) # gato/perro
y confiable
    return b.xyxy.cpu().numpy()[keep].astype(np.float32)        # (N, 4) en
XYXY
```

`.cpu().numpy()` pasa los datos de la GPU a un arreglo de numpy. `keep` es una **máscara booleana**: True en las cajas que cumplen las dos condiciones.

2. `sam_refine`: un encoder, todas las cajas en un lote.

```
def sam_refine(predictor, image_rgb, boxes):
    predictor.set_image(image_rgb)                              #
encoder ViT: UNA vez
    b = torch.as_tensor(boxes, dtype=torch.float32, device=predictor.device)
    b = predictor.transform.apply_boxes_torch(b, image_rgb.shape[:2])  #
cajas al espacio 1024 de SAM
    masks, _, _ = predictor.predict_torch(point_coords=None,
point_labels=None,
                                          boxes=b, multimask_output=False)  #
todas a la vez
    return masks[:, 0].any(0).cpu().numpy()                     #
unión -> (H, W) bool
```

- SAM trabaja internamente a 1024 px, así que las cajas, que están en píxeles de la foto original, se **reescalan** con `apply_boxes_torch`.
- `predict_torch` recibe todas las cajas juntas (un **lote**): una sola llamada al decoder en vez de una por caja.
- `multimask_output=False` pide una sola máscara por caja. `masks` tiene forma `(N, 1, H, W)`; `masks[:, 0]` quita el 1 y `.any(0)` hace la **unión** (un píxel es mascota si alguna máscara lo marca).

3. **`fallback_mask` : qué hacer si YOLO no ve ningún gato o perro.** Tres pasos en cascada:

```
def fallback_mask(image_rgb):
    # 1) reintentar con umbral bajo, solo cat/dog
    r = yolo.predict(to_bgr(image_rgb), conf=0.05)[0]
    boxes = yolo_pet_boxes(r, conf_thres=0.05)
    # 2) la caja más confiable de cualquier animal de COCO o peluche
    if len(boxes) == 0 and r.boxes is not None and len(r.boxes):
        cls = r.boxes.cls.cpu().numpy().astype(int)
        animal = ((cls >= 14) & (cls <= 23)) | (cls == 77)        # pájaro…
jirafa, oso de peluche
        if animal.any():
            i = int(np.argmax(np.where(animal, r.boxes.conf.cpu().numpy(),
-1)))
            boxes = r.boxes.xyxy.cpu().numpy()[i:i + 1]
    # 3) último recurso: caja central
    if len(boxes) == 0:
        H, W = image_rgb.shape[:2]
        boxes = np.array([[0.15 * W, 0.10 * H, 0.85 * W, 0.95 * H]])
    return sam_refine(predictor, image_rgb, boxes)
```

- En YOLO las clases 14 a 23 de COCO son animales (pájaro, gato, perro, caballo, oveja, vaca, elefante, oso, cebra, jirafa) y la 77 es el oso de peluche.
- `np.where(animal, conf, -1)` pone −1 a lo que no es animal, y `argmax` elige el animal más confiable.
- La caja central (15–85 % del ancho, 10–95 % del alto) es un **prior general** de Pets, donde la mascota suele estar centrada. No se ajustó mirando estas imágenes, así que respeta las reglas.
- Ningún paso usa el trimap.

4. **`challenge_pipeline` : une todo.**

```
def challenge_pipeline(image_rgb):
    r = yolo.predict(to_bgr(image_rgb), retina_masks=True, conf=0.05)[0]
    boxes = yolo_pet_boxes(r)                    # filtro final a 0.25
    return sam_refine(predictor, image_rgb, boxes) if len(boxes) else
fallback_mask(image_rgb)
```

YOLO corre con `conf=0.05` para no perder detecciones, pero `yolo_pet_boxes` filtra a 0.25 por defecto, el mismo umbral que la referencia. Si hay cajas, se refinan; si no, entra el fallback.

Evaluación (celda que no se modifica)

Corre `run_benchmark` dos veces, con `yolo_pet_mask` y con `challenge_pipeline`, y compara imagen por imagen:

Método	IoU	Dice	Sin detección	ms/img
YOLO26n-seg solo	0.9419	0.9541	3 %	19
YOLO → SAM (nuestro)	**0.9523**	**0.9709**	**0 %**	469

`ganancia media de IoU: +0.0104 | imágenes donde ganas: 46% | empatas: 3% | SUPERADO`

Parte B: figuras y entrega

Figura de ganancias y pérdidas (columnas: verdad, YOLO, YOLO → SAM):

Fila	Foto	YOLO	Pipeline	Δ	Qué pasó
1	perro en primer plano (pierde)	0.99	0.30	−0.69	la caja cubre toda la foto; SAM marca el pecho y el suelo, y deja fuera la cara
2	gato en primer plano (pierde)	0.94	0.53	−0.41	SAM pinta la pared de fondo y deja huecos en la cara
3	gato siamés en una rama (gana)	0.00	1.00	+1.00	YOLO no vio gato; el fallback lo rescató
4	perro negro (gana)	0.00	0.99	+0.99	ídem

Imagen propia: la pomerania (`/content/pomerania.png`). La máscara cubre todo el perro, incluido el pelaje esponjoso del contorno; solo quedan pequeños huecos en ojos y nariz (regiones oscuras que SAM separa) y unos puntos sueltos junto a una pata. Demuestra que el pipeline funciona fuera del dataset.

El archivo de entrega `reto_s2_2235112.pt` guarda: código estudiantil, semilla, índices de las 100 imágenes, las 100 máscaras comprimidas (`np.packbits`, 8 píxeles por byte), las métricas de ambos métodos, `passed: True`, el código fuente de nuestras tres funciones (`inspect.getsource`), versiones de pesos y torch, la imagen propia y la fecha.

La huella SHA-256 (`97ef26c2…`) es un resumen criptográfico de todas las máscaras: si cambia un solo píxel, cambia la huella entera. Sirve para que el profesor verifique que las máscaras entregadas son las que produce nuestro código. Salió **idéntica en todas**

nuestras ejecuciones, así que el pipeline es reproducible (determinista).

Diagnóstico y respuestas del informe

Agregamos una celda de diagnóstico:

```
img 24 | paso 2: animal COCO 'sheep' | IoU YOLO 0.000 -> pipeline 0.989
img 32 | paso 2: animal COCO 'bear'  | IoU YOLO 0.000 -> pipeline 0.996
img 80 | paso 2: animal COCO 'bear'  | IoU YOLO 0.000 -> pipeline 0.990
mediana ms | YOLO 18.9 | SAM encoder (set_image) 392.8 | SAM decoder 13.9
```

1. ¿En qué imágenes pierde? Gana en 46, empata en 3 (todas con IoU 1.00) y pierde en 51, pero de forma asimétrica: cuando gana es por +0.013 de media y cuando pierde, por −0.049. Las medianas son casi iguales (0.9930 vs 0.9939): en la foto típica SAM no cambia nada. De las 51 pérdidas, 31 son menores a 0.01 (unos pocos píxeles de borde), 8 están entre 0.01 y 0.05 y solo 12 superan 0.05. Las 4 peores (−0.690, −0.408, −0.275, −0.227) restan −0.016 por sí solas. Las pérdidas grandes son **primeros planos**: la caja de YOLO cubre casi toda la imagen, el prompt se vuelve ambiguo y SAM, obligado a dar una sola máscara, confunde objeto y fondo. Y como YOLO ya era casi perfecto ahí, cualquier error de SAM es pérdida.

2. ¿Cuántos ms añade SAM y compensa? Pasa de 18.8 a 468.5 ms/img: **unos 450 ms más, ~25 veces más lento**. El desglose: YOLO 18.9 ms, encoder de SAM **392.8 ms (97 % del costo de SAM)**, decoder 13.9 ms. A cambio: +0.0104 de IoU y +0.0168 de Dice. **No compensa en tiempo real** (de ~50 a ~2 imágenes por segundo). Un diseño mejor sería usar SAM **solo cuando YOLO no detecta**: con los números por imagen de esta ejecución daría IoU ≈ 0.972 con ~33 ms/img (análisis a posteriori, no la entrega evaluada).

3. ¿Qué hizo el fallback y cuánto aportó? Se activó en 3 imágenes, justo el 3 % sin detección de YOLO. **Las tres se resolvieron en el paso 2**: YOLO sí había localizado al animal, pero lo etiquetó como *sheep* o *bear*. Es decir, YOLO no falló en localizar sino en **clasificar**, y a SAM no le importa la clase, solo dónde mirar. Obtuvo IoU 0.989, 0.996 y 0.990, lo que aporta **+0.0298** al IoU medio. Ese aporte es **mayor que la ganancia total** (+0.0104): en las 97 imágenes donde YOLO sí detectó, SAM baja el IoU de 0.9711 a 0.9511 (−0.0199). Sin el fallback, el pipeline quedaría en 0.9225 y **no superaría el reto**.

Preguntas probables del profesor

Respuestas cortas, para decir en voz alta. Cada una cabe en 20–30 segundos.

Sobre el reto

¿Por qué su pipeline supera a YOLO si SAM empeora la mayoría de las máscaras? Porque el fallback recupera 3 mascotas que YOLO no detectó como gato o perro. Esas 3 imágenes pasan de IoU 0 a ~0.99 y aportan +0.0298. En las 97 restantes SAM resta −0.0199. El neto es +0.0104.

¿Entonces SAM no sirve? Sirve para lo que es bueno: recortar cuando alguien le dice dónde mirar. Lo que no aporta es refinar máscaras que YOLO ya hace casi perfectas. Su valor apareció cuando la caja venía de una detección mal clasificada: SAM no necesita la clase.

¿Por qué pierde tanto en los primeros planos? Porque la caja de YOLO cubre casi toda la foto. Una caja así no le dice a SAM qué es objeto y qué es fondo, y con `multimask_output=False` tiene que elegir una sola interpretación. Ya lo vimos en la sección 4.4: con la caja que cubría todo el gato blanco, SAM metió la manta (IoU 0.80).

¿Hicieron trampa con la caja central o los umbrales? No. Ningún paso usa el trimap. Los umbrales (0.25 y 0.05) y la caja central (15–85 %, 10–95 %) se fijaron antes de evaluar, como priors generales; además, la caja central nunca se activó.

¿Qué significa la huella SHA-256? Es un resumen único de las 100 máscaras. Si cambia un píxel, cambia la huella. Permite verificar que el `.pt` corresponde a nuestro código. Salió igual (`97ef26c2…`) en todas las ejecuciones: el resultado es reproducible.

¿Por qué `set_image` una sola vez y las cajas en lote? Porque el encoder cuesta 392.8 ms y el decoder 13.9 ms. Si hay dos mascotas, correr el encoder dos veces duplicaría el costo sin ganar nada. En lote, todas las cajas pasan juntas por el decoder.

¿Por qué `apply_boxes_torch` ? SAM redimensiona la imagen a 1024 px por su lado largo. Las cajas están en píxeles de la foto original y hay que llevarlas a esa misma escala, o caerían en el lugar equivocado.

¿Por qué `to_bgr` ? Ultralytics asume que un arreglo de numpy viene en BGR, el orden de OpenCV. Nuestras imágenes son RGB; sin convertir, YOLO vería los colores invertidos y detectaría peor.

¿Cómo mejorarían el pipeline? Usar SAM solo cuando YOLO no detecta un gato o perro, y quedarse con la máscara de YOLO en el resto. Con nuestros números daría IoU ≈ 0.972 a ~33 ms/img: más preciso y unas 14 veces más rápido. Otra mejora: combinar caja + puntos en el prompt de SAM.

Sobre conceptos

¿Qué diferencia hay entre IoU y Dice? Ambos miden solapamiento. IoU = intersección / unión; Dice = 2 × intersección / (suma de áreas). Dice siempre es mayor o igual y se relaciona así: Dice = 2·IoU / (1 + IoU).

¿Por qué se ignora el borde del trimap? Porque es una franja dudosa donde ni el anotador sabe si el píxel es mascota o fondo. Contarla castigaría a los modelos por algo ambiguo.

¿Semántica vs instancias? Semántica dice qué clase tiene cada píxel (dos perros son la misma mancha "perro"). Instancias separa cada objeto (perro 1 y perro 2). U-Net hace semántica; Mask R-CNN y YOLO-seg, instancias.

¿Qué es mask AP? Precisión promedio sobre umbrales de IoU de 0.50 a 0.95. AP50 solo exige IoU ⩾ 0.5; AP75 exige máscaras muy ajustadas. Una brecha grande entre AP50 y AP75 indica que el modelo encuentra los objetos pero los recorta mal.

¿Por qué Mask R-CNN tiene un techo? Porque toda máscara se describe con 28 × 28 = 784 valores. Medimos que eso limita el IoU a 0.9779 aunque la red fuera perfecta.

¿Cómo funciona YOLO-seg? Genera 32 prototipos (máscaras base) para toda la imagen y, por cada objeto, 32 coeficientes. La máscara es $\sigma(P \cdot c)$ recortada a la caja. Por eso es tan rápido: cada objeto cuesta un producto matriz–vector.

¿Por qué SAM da 3 máscaras? Por ambigüedad: un punto en la camisa puede ser la camisa, el torso o la persona. En la sección 4.4, la máscara con mayor IoU predicho (0.97) fue el ojo, con IoU real 0.01: el puntaje mide calidad de forma, no si es lo que queríamos.

¿U-Net++ es mejor que U-Net? Por poco: mIoU 0.7184 vs 0.7085, sobre todo en el borde. Pero es una sola semilla, y es un 44 % más lenta aunque tenga menos parámetros. La supervisión profunda no mejora la salida final: permite podar.

¿Por qué los modelos de COCO ganan a la U-Net en Pets si nunca vieron Pets? COCO tiene miles de gatos y perros, y esos modelos son mucho más grandes y trabajan a mayor resolución. La U-Net del taller tiene 1.7 M parámetros y trabaja a 128 px.

¿YOLO o Mask R-CNN para peatones? YOLO para tiempo real: 6.7× más rápido y 15× más pequeño por solo −1.9 puntos de mask AP. Mask R-CNN si importa más encontrar cada instancia (box AP +6 puntos).

¿Por qué el SAM con caja oráculo sacó solo 0.879? Porque `gt_box` incluye los píxeles de borde del trimap, y en dos fotos esos píxeles estaban lejos de la mascota: la caja se infló hasta cubrir casi toda la imagen. Con puntos sobre la mascota, esas mismas fotos dan 0.99. Ese 0.879 está subestimado.

Guion de sustentación

Unos 10 minutos en total. Lo que va entre comillas es para decirlo; lo que va en cursiva es qué mostrar en el notebook. Si la sustentación es individual, cada quien puede llevar todo el guion; si es grupal, aquí va una repartición sugerida por bloques.

1. Apertura (1 min) — integrante 1

"Buenos días. Somos Gustavo Chipantiza, Federico Gonzales, Juan Jacobo Delgado y María de los Ángeles Valencia. Nuestro taller compara cuatro formas de segmentar imágenes: U-Net++, Mask R-CNN, YOLO-seg y SAM, y termina con un reto: combinar YOLO y SAM para superar a YOLO solo."

"Usamos dos datasets: Oxford-IIIT Pet, con una mascota por foto, para comparar todos los modelos en las mismas 100 imágenes; y PennFudanPed, con 170 fotos y 423 peatones, para entrenar segmentación de instancias."

Mostrar: la tabla del mapa de la sesión y el histograma de peatones por imagen.

2. Los cuatro modelos (2 min) — integrante 2

"U-Net++ mejora la U-Net con conexiones anidadas: cada nodo recibe todos los anteriores de su nivel. Con anchos 16 y 32, por ejemplo, el nodo X^{0,3} recibe 3·16 + 32 = 80 canales."

"Mask R-CNN primero detecta cajas y luego dibuja una máscara de 28 por 28 dentro de cada una. Medimos que esa resolución pone un techo de IoU de 0.978."

"YOLO-seg genera 32 prototipos y cada objeto los mezcla con sus coeficientes. En el ejemplo de clase, dos prototipos opuestos con coeficientes 1 y 1 se cancelan y dan 0.5 en todos lados."

"SAM no sabe qué es un objeto, solo lo recorta si le dices dónde. Con un punto en el ojo del gato propuso tres máscaras; la más confiada, 0.97, era solo el ojo, con IoU real 0.01."

Mostrar: los prototipos de YOLO y los cinco paneles de SAM.

3. Resultados de las secciones 5 a 7 (2 min) — integrante 3

"Reentrenamos Mask R-CNN para peatones cambiando sus cabezas de 91 a 2 clases. En 8 épocas llegó a mask AP 0.829 en test. Las curvas muestran que desde la primera época ya encontraba a los peatones, AP50 de 0.98; lo que mejoró fue el contorno."

"Sin entrenar nada, en Pets Mask R-CNN logró IoU 0.972 y YOLO 0.942. Pero el boxplot muestra que YOLO no tiene máscaras malas: su promedio baja por 3 fotos donde no detectó nada. Esa observación es la base del reto."

"U-Net++ superó a U-Net por solo 0.01 de mIoU y es 44 % más lenta. La poda no conviene: la U-Net completa es casi igual de rápida y mejor. Y YOLO reentrenado quedó a 1.9 puntos de Mask R-CNN siendo 6.7 veces más rápido."

Mostrar: la tabla 7.1 y la gráfica de distribución del IoU por imagen.

4. El reto (3 min) — integrante 4

"El pipeline es: YOLO detecta las cajas de gato o perro, SAM refina cada caja y unimos las máscaras. Implementamos tres funciones."

"`yolo_pet_boxes` filtra las cajas por clase y confianza mayor o igual a 0.25. `sam_refine` corre el encoder de SAM una sola vez y pasa todas las cajas en un lote, después de escalarlas al espacio de 1024 de SAM. `fallback_mask` actúa si YOLO no encuentra ningún gato o perro: primero reintenta con umbral 0.05, luego toma la caja de cualquier animal de COCO, y como último recurso una caja central. Nada usa la verdad."

"Resultado: IoU 0.9523 contra 0.9419, ganancia de +0.0104. Superado."

Mostrar: la salida de la evaluación y la figura de ganancias y pérdidas.

5. Lo que descubrimos (1.5 min) — integrante 4 o todos

"Lo más interesante es de dónde sale la ganancia. Con el diagnóstico vimos que el fallback se activó en 3 fotos y las 3 se resolvieron en el paso 2: YOLO sí había visto al animal, pero lo llamó oveja u oso. SAM no necesita la clase, solo dónde mirar, y sacó 0.99 en las tres. Eso aporta +0.030."

"En cambio, en las 97 fotos donde YOLO sí acertó, SAM resta 0.020, sobre todo en primeros planos, donde la caja cubre toda la foto y SAM confunde objeto y fondo. Sin el fallback, no habríamos superado el reto."

"Y el costo: SAM sube la latencia de 19 a 469 milisegundos; el 97 % es su encoder. Proponemos usar SAM solo cuando YOLO no detecta: con nuestros números daría IoU de 0.972 a unos 33 milisegundos."

Mostrar: la salida del diagnóstico (sheep, bear, bear) y la imagen de la pomerania.

6. Cierre (30 s)

"En resumen: cada modelo tiene su lugar. YOLO es el más rápido, Mask R-CNN el más preciso entre los que conocen clases, y SAM recorta muy bien si alguien le dice dónde mirar. Combinarlos funciona, pero no por refinar contornos sino por rescatar lo que el detector clasifica mal. Entregamos el notebook y el archivo `reto_s2_2235112.pt`, con huella reproducible. Gracias."

Números que hay que saber de memoria

Qué	Valor
IoU YOLO solo / pipeline	0.9419 / 0.9523 (+0.0104)
Dice YOLO solo / pipeline	0.9541 / 0.9709
Latencia YOLO / pipeline	19 / 469 ms por imagen
Encoder / decoder de SAM	392.8 / 13.9 ms
Aporte del fallback	+0.0298 (3 imágenes, paso 2: sheep, bear, bear)
Efecto de SAM en las 97 detectadas	−0.0199
Gana / empata / pierde	46 / 3 / 51 imágenes
Mask R-CNN en peatones	mask AP 0.829
Techo de máscara 28 × 28	IoU 0.9779
U-Net vs U-Net++ (mIoU)	0.7085 vs 0.7184
YOLO vs Mask R-CNN en peatones	18.2 vs 122.1 ms; mask AP 0.810 vs 0.829
Huella	`97ef26c2…`